
Security Review Report
NM-1052 - VANA Staking



NETHERMIND
SECURITY

(September 29, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	Architectural Components	4
4.2	Reward Ingestion, Conversion, and Distribution	4
4.3	Entity Vesting Models	4
4.4	Staking, Bonding, Unstaking, and Redelegating	4
5	Risk Rating Methodology	5
6	Issues	6
6.1	[Critical] A rigged STREAM entity can drain other entities' principal from the shared VanaPoolTreasury	6
6.2	[High] A malicious entity owner can capture or withhold the rewards the splitter has already paid to their entity's stakers	9
6.3	[Medium] A large matured position nullifies the bonding period for subsequent deposits	10
6.4	[Medium] Accrued commission can never be withdrawn from the treasury	11
6.5	[Medium] Redelegating into a matured position makes already earned rewards forfeitable	12
6.6	[Medium] Reward distribution weight can be increased by settling rewards early	13
6.7	[Medium] Rounding dust left by the final unstake disables an entity	14
6.8	[Medium] The executeBuyAndBurn(...) function sends tokens to a configurable sink address instead of destroying them	15
6.9	[Medium] The entity registration stake never serves a bonding period	16
6.10	[Low] A one wei deposit can capture an entire elapsed reward stream	17
6.11	[Low] Switching reward models leaves a stale stream schedule installed	18
6.12	[Low] The VANA denominated unstake disables the minimum output check	19
6.13	[Low] The computed bonding deadline is bounded by neither the configured period nor its maximum	20
6.14	[Low] The maximum unstake view simulates only the APY reward model	21
6.15	[Low] Unclamped decrement of distributed rewards blocks early unstaking	22
6.16	[Info] Redelegating can revert when the cost basis exceeds the share value	22
6.17	[Info] The _vestStream(...) function casts the timestamp to a variable that will overflow in the future	23
6.18	[Info] The committedRewards(...) view can report a stale value before rewards are settled	24
6.19	[Info] The configured minimum stake amount is never enforced	25
6.20	[Info] The max amount quoted by getMaxUnstakeAmount(...) can be rejected by unstakeVana(...) after the minimum-stake-on-exit rule	26
6.21	[Info] Unstaking cost grows linearly with the number of entities ever created	27
6.22	[Best Practice] The receive() function doesn't check if msg.value = 0	27
6.23	[Best Practice] The updateEntityPool(...) function contains a redundant role check	28
6.24	[Best Practice] The codebase contains unused error and event declarations	28
6.25	[Best Practices] The calculateYield(...) interface declares its first two parameters in the wrong order	29
6.26	[Best Practices] The buy and burn sink can be cleared while a reserve is still outstanding	29
7	Documentation Evaluation	30
8	Test Suite Evaluation	31
8.1	Tests Output	31
8.2	Automated Tools	33
8.2.1	AuditAgent	33
9	About Nethermind	34

1 Executive Summary

This document presents the results of a security review conducted by [Nethermind Security](#) for [Vana's](#) staking and reward distribution mechanism. These contracts implement a delegated staking layer in which users deposit native VANA tokens to back specific entities, earning rewards proportional to their delegated stake over time. The protocol manages entity pools, supports offchain ERC20 reward conversions, applies operator commissions, and delegates the vesting of locked rewards to either continuous compounding (APY) or linear streaming (STREAM) models.

The system is designed so that all primary user interactions, bonding period enforcement, and cross-entity redelegations reside within the VanaPoolStaking contract, while entity lifecycle and reward accounting are defined in the VanaPoolEntity contract. The RewardSplitter determines each entity's protocol reward allocation by measuring its accrued staked principal, optionally applying a buy-and-burn protocol fee before funding the entities. As part of this engagement, the upgradeable proxy architecture and the core functional logic of these components have been reviewed.

The audit comprises 1485 lines of Solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools. **Along this document, we report** 26 points of attention, where one is classified as Critical, one is classified as High, seven are classified as Medium, six are classified as Low, six are classified as Informational and five are classified as Best Practice severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the system overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

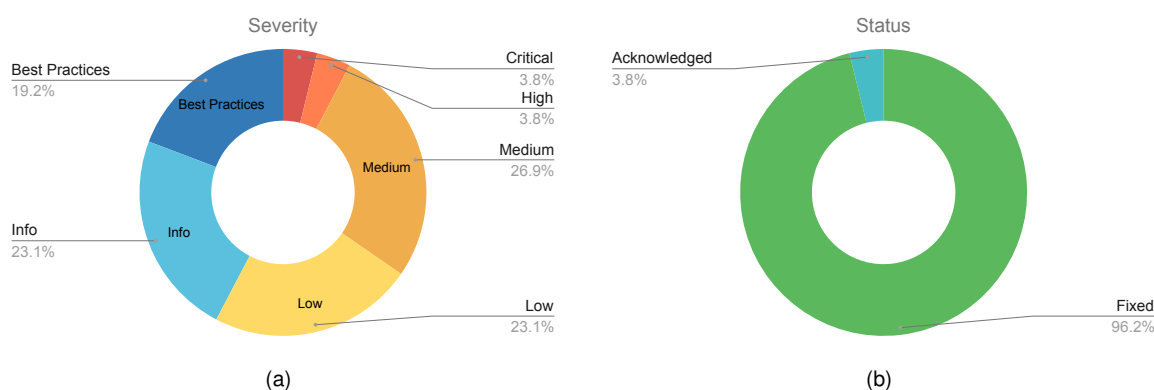


Fig. 1: Distribution of issues: Critical (1), High (1), Medium (7), Low (6), Undetermined (0), Informational (6), Best Practices (5). Distribution of status: Fixed (25), Acknowledged (1), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	September 29, 2026
Final Report	September 29, 2026
Initial Commit	e6c258a
Final Commit	e04a872
Documentation Assessment	High
Test Suite Assessment	Medium

2 Audited Files

	File	LoC	Comments	Ratio	Blank	Total
1	vanaStaking/vanaPoolTreasury/VanaPoolTreasuryImplementation.sol	38	2	5.3%	13	53
2	vanaStaking/vanaPoolTreasury/VanaPoolTreasuryProxy.sol	5	1	20.0%	2	8
3	vanaStaking/vanaPoolTreasury/interfaces/VanaPoolTreasuryStorageV1.sol	5	6	120.0%	2	13
4	vanaStaking/vanaPoolTreasury/interfaces/IVanaPoolTreasury.sol	7	1	14.3%	1	9
5	vanaStaking/rewardSplitter/RewardSplitterProxy.sol	5	1	20.0%	2	8
6	vanaStaking/rewardSplitter/RewardSplitterImplementation.sol	209	86	41.1%	38	333
7	vanaStaking/vanaPoolStaking/VanaPoolStakingProxy.sol	5	1	20.0%	2	8
8	vanaStaking/vanaPoolStaking/VanaPoolStakingImplementation.sol	463	267	57.7%	133	863
9	vanaStaking/vanaPoolStaking/interfaces/IVanaPoolStaking.sol	39	1	2.6%	26	66
10	vanaStaking/vanaPoolStaking/interfaces/VanaPoolStakingStorageV2.sol	5	6	120.0%	2	13
11	vanaStaking/vanaPoolStaking/interfaces/VanaPoolStakingStorageV1.sol	12	6	50.0%	5	23
12	vanaStaking/vanaPoolEntity/VanaPoolEntityProxy.sol	5	1	20.0%	2	8
13	vanaStaking/vanaPoolEntity/VanaPoolEntityImplementation.sol	575	418	72.7%	163	1156
14	vanaStaking/vanaPoolEntity/interfaces/IVanaPoolEntity.sol	97	20	20.6%	48	165
15	vanaStaking/vanaPoolEntity/interfaces/VanaPoolEntityStorageV1.sol	15	7	46.7%	8	30
	Total	1485	824	55.5%	447	2756

3 Summary of Issues

	Finding	Severity	Update
1	A rigged STREAM entity can drain other entities' principal from the shared VanaPoolTreasury	Critical	Fixed
2	A malicious entity owner can capture or withhold the rewards the splitter has already paid to their entity's stakers	High	Fixed
3	A large matured position nullifies the bonding period for subsequent deposits	Medium	Acknowledged
4	Accrued commission can never be withdrawn from the treasury	Medium	Fixed
5	Redelegating into a matured position makes already earned rewards forfeitable	Medium	Fixed
6	Reward distribution weight can be increased by settling rewards early	Medium	Fixed
7	Rounding dust left by the final unstake disables an entity	Medium	Fixed
8	The executeBuyAndBurn(...) function sends tokens to a configurable sink address instead of destroying them	Medium	Fixed
9	The entity registration stake never serves a bonding period	Medium	Fixed
10	A one wei deposit can capture an entire elapsed reward stream	Low	Fixed
11	Switching reward models leaves a stale stream schedule installed	Low	Fixed
12	The VANA denominated unstake disables the minimum output check	Low	Fixed
13	The computed bonding deadline is bounded by neither the configured period nor its maximum	Low	Fixed
14	The maximum unstake view simulates only the APY reward model	Low	Fixed
15	Unclamped decrement of distributed rewards blocks early unstaking	Low	Fixed
16	Redelegating can revert when the cost basis exceeds the share value	Info	Fixed
17	The _vestStream(...) function casts the timestamp to a variable that will overflow in the future	Info	Fixed
18	The committedRewards(...) view can report a stale value before rewards are settled	Info	Fixed
19	The configured minimum stake amount is never enforced	Info	Fixed
20	The max amount quoted by getMaxUnstakeAmount(...) can be rejected by unstakeVana(...) after the minimum-stake-on-exit rule	Info	Fixed
21	Unstaking cost grows linearly with the number of entities ever created	Info	Fixed
22	The receive() function doesn't check if msg.value = 0	Best Practices	Fixed
23	The updateEntityPool(...) function contains a redundant role check	Best Practices	Fixed
24	The codebase contains unused error and event declarations	Best Practices	Fixed
25	The calculateYield(...) interface declares its first two parameters in the wrong order	Best Practices	Fixed
26	The buy and burn sink can be cleared while a reserve is still outstanding	Best Practices	Fixed

4 System Overview

The Vana Staking contracts are a staking and reward distribution mechanism designed to manage entity pools, user staking, dynamic yield allocation, and treasury custody. Users participate by staking native tokens to specific entities, earning rewards proportional to their delegated stake over time. The system employs a modular smart contract architecture utilizing upgradeable proxies to manage fund flows, entity lifecycles, and reward conversions.

4.1 Architectural Components

The protocol's core logic is distributed across four primary smart contracts:

- **VanaPoolStaking**: The main user-facing contract where stakers deposit funds, mint or burn entity shares, and manage their staking positions.
- **VanaPoolEntity**: Manages the lifecycle of staking entities, maintains the accounting of the `lockedRewardPool` and `activeRewardPool`, and tracks cumulative staked principal.
- **RewardSplitter**: Responsible for splitting a protocol reward budget across registered entities in proportion to their accrued staked principal.
- **VanaPoolTreasury**: Serves as the central native asset vault, holding VANA tokens and executing permissioned fund transfers controlled by the staking and entity contracts.

4.2 Reward Ingestion, Conversion, and Distribution

The protocol supports the ingestion of both native VANA and ERC20 token rewards. While native VANA is funded directly into the `RewardSplitter` contract, ERC20 rewards are held in a `pendingConversion` state until a maintainer forwards them to a trusted offchain converter. This converter swaps the tokens into native VANA and returns the funds to the splitter for distribution.

During a distribution round, the `RewardSplitter` calculates each entity's weight based on its accrued staked principal. The contract then proportionally splits the active budget among the eligible entities. Additionally, the system incorporates a buy-and-burn mechanism, allowing a configurable percentage of the distributed budget (the `burnRate`) to be skimmed and sent to `address(0)` for burning native tokens, before the remaining funds are allocated to entities.

4.3 Entity Vesting Models

Once rewards are allocated to an entity, they are placed in its `lockedRewardPool`. The entity then vests these funds into the `activeRewardPool` to benefit its stakers, utilizing one of two configurable `RewardModel` structures:

- **APY Model**: Applies a continuous, compounding yield drip up to a defined `maxAPY`.
- **STREAM Model**: Utilizes a linear vesting schedule to distribute scheduled rewards over a fixed duration, with support for queuing a single follow-on reward stream.

Regardless of the chosen model, operators can configure a `commissionRate`. Whenever rewards are processed, the operator's commission is skimmed from the vested amount and credited to their claimable `accruedCommission` balance. The remainder flows into the `activeRewardPool` to proportionally raise the share price for the entity's delegators.

***All pool entities are planned to migrate over to a STREAM model.**

4.4 Staking, Bonding, Unstaking, and Redelegation

Users interact with the protocol by staking VANA into specific entities in exchange for proportional shares. The system enforces a `bondingPeriod` to encourage long term participation and stability.

- **Staking and Bonding**: When a user stakes, their `rewardEligibilityTimestamp` is updated based on a value weighted average of their existing remaining bond and the new stake's bonding period.
- **Unstaking**: If a user unstakes after their bonding period matures, they receive their full share value, which includes both their principal and accrued rewards. However, if they unstake early during the bonding period, they forfeit their pending rewards and only receive their proportional principal, known as the `costBasis`. Forfeited rewards are immediately returned to the entity's `lockedRewardPool` for future redistribution.
- **Anti-Gaming Mechanisms**: To prevent wash attacks where old capital accelerates the vesting of new capital, partial unstaking during the bonding period dynamically extends the remaining bonding time for the staker's leftover balance.
- **Redelegation**: Users can seamlessly move their staking positions between entities without triggering a full withdrawal. Redelegating transfers the user's full share value, cost basis, and remaining bonding time to the new entity, ensuring that accumulated rewards are preserved provided the original bond maturity is eventually met in the new entity.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Critical] A rigged STREAM entity can drain other entities' principal from the shared VanaPoolTreasury

File(s): `/vanaPoolStaking/VanaPoolStakingImplementation.sol, /vanaPoolEntity/VanaPoolEntityImplementation.sol`

Description: Every VanaPool entity keeps its funds in one shared VanaPoolTreasury contract, and each entity is a share based pool: `activeRewardPool` records the VANA it owns and `totalShares` records the claims against it. Entities are created by a maintainer on behalf of a third party operator, who becomes the entity owner and controls that entity's reward scheduling. A staker who exits before their bonding period matures is returned their principal and forfeits accrued rewards.

Shares are not minted in a single division. The `stake(...)` function of the VanaPoolStaking contract first asks the VanaPoolEntity contract for a rate, computed by the `vanaToEntityShare(...)` function as $(totalShares * 1e18) / activeRewardPool$, and then applies that rate to the deposit:

```
1 uint256 vanaToShare = vanaPoolEntity.vanaToEntityShare(entityId);
2 // @audit-issue The rate returned above is already floored, so the depositor
3 // receives floor(r) / r of the shares paid for, not a single wei less.
4 uint256 sharesIssued = (vanaToShare * stakeAmount) / 1e18;
```

Because the truncation lands on the rate rather than on the result, a depositor receives $\text{floor}(r) / r$ of the shares they paid for. For an ordinary entity the rate is around $1e18$ and the loss is invisible. When the true rate sits just below an integer m , the depositor receives only $(m - 1) / m$ of their shares, which is as little as half at $m = 2$. Their wei still enters `activeRewardPool`, so the shortfall silently accrues to whoever already holds shares.

An entity owner can move their own entity into that regime using only owner level calls. The registration stake written by the `registerEntityStake(...)` function never sets `rewardEligibilityTimestamp`, so it is unbonded from birth and the owner can withdraw all but a chosen dust share count. The owner then calls the `distributeRewards(...)` function, which is owner callable on a STREAM entity, to vest VANA into `activeRewardPool` without minting any shares. Leaving S_0 shares against a parked amount P fixes the rate at $S_0 * 1e18 / P$, so the owner selects m directly. This is not reachable in APY mode, where the drip computed by the `calculateYield(...)` function is proportional to `activeRewardPool` and a dust pool therefore drips dust.

The second half of the problem is that a stake made while the position is already inside its bonding period is booked at `raw msg.value` rather than at the value of the shares actually minted. The branch is selected by reading `rewardEligibilityTimestamp` as it was stored before the call, so a brand new address takes the eligible branch on its first ever deposit and is booked at value; it becomes bonded only as that same call returns. Raw wei treatment therefore applies from the second deposit onwards, and a position that is fully exited keeps its `rewardEligibilityTimestamp`, so the bond survives indefinitely once armed:

```
1 } else {
2   // @audit-issue The cost basis grows by raw wei, while the shares bought
3   // with those wei are under-issued by the floored rate.
4   stakerEntity.costBasis += stakeAmount;
5   // ...
6 }
```

An early exit then refunds that raw cost basis, while the entity's books are reduced by only the smaller share value. Both happen inside the `_unstake(...)` function:

```
1 } else {
2   // @audit-issue The treasury pays out the raw cost basis.
3   vanaToReturn = proportionalCostBasis;
4   // ...
5 }
6
7 // ...
8
9 // @audit The entity gives up only shareValue, which is less than the refund.
10 vanaPoolEntity.updateEntityPool(entityId, shareAmount, shareValue, false);
```

The attacker therefore dilutes themselves inside their own entity and is then refunded pre-dilution. Because all entities share one treasury, the difference is paid out of other entities' deposits. One cycle at $m = 2$ against $1000e18$ wei of parked capital proceeds as follows:

1. The entity is created with the $1e18$ wei registration stake, minting $1e18$ shares against an `activeRewardPool` of $1e18$ wei. The rate is $1e18$ and the entity behaves normally;
2. The owner unstakes all but 2000 shares. The registration position is unbonded, so this succeeds immediately and leaves 2000 shares against 2000 wei;

3. The owner calls the `distributeRewards(...)` function with $1000e18$ wei of their own VANA and a duration of 0, and the next call to the `processRewards(...)` function moves it into `activeRewardPool`. The entity now holds $1000e18 + 2000$ wei against 2000 shares, so the true rate is 1.999999999999999996 , the `vanaToEntityShare(...)` function returns 1, and one share is worth $5e17$ wei;
4. A second address controlled by the attacker makes a seed deposit of $1e18$ wei. Its stored `rewardEligibilityTimestamp` is still 0 when the branch is chosen, so this first deposit takes the eligible branch, is booked at the value of the shares minted, and opens no gap. Its only purpose is to leave the position bonded, which the same call does before returning;
5. The same address then deposits $1414e18$ wei. This time the branch test reads a `rewardEligibilityTimestamp` in the future, so `costBasis` grows by the raw $1414e18$ wei to a total of $1414.5e18$ wei, while the 1415 shares held are worth only about $1000.65e18$ wei;
6. That address exits before the bond matures. The treasury refunds the raw $1414.5e18$ wei while the entity's books give up only the $1000.65e18$ wei of share value. Treasury cash is barely moved across the deposits and the refund, but the roughly $414e18$ wei difference stays in `activeRewardPool`, backed by no cash and owned entirely by the owner's 2000 shares;
7. The owner's dust position is therefore worth about $1414.35e18$ wei instead of the $1000e18$ wei that was parked. When the owner finally unstakes it, the treasury pays out that full amount, and the roughly $413.85e18$ wei of net gain is real cash belonging to the stakers of other entities;

The impacts are the following:

- **Any honest depositor into such an entity loses up to 50% of their deposit** to the entity owner's dust share position, on a single ordinary `stake(...)` call. The `shareAmountMin` parameter is the only defence and is opt-in;
- **The shortfall is taken from other entities' stakers.** No honest counterparty is required, since both the dust position and the depositing address belong to the attacker. Each cycle deposits b and refunds b , so treasury cash is flat while the entity's books inflate; the cash leaves only when the owner finally unstakes the dust position, and it is paid out of other entities' principal;
- **The total is not bounded by the attacker's capital.** Repeating the cycle walks the rate from m down to 1, and the ceiling per entity is $\text{park} * (m - 1)$. Since m is chosen freely by how far the registration stake is drained, bounded only by $50 \leq 1e18$, there is no meaningful limit in code. The practical cap is the honest TVL present in the treasury;
- **The treasury becomes insolvent**, not merely short. Once drained, honest stakers calling `unstake(...)` revert with `TransferFailed` because the `transferVana(...)` function cannot pay what the books promise;
- **The parked capital need not belong to the attacker.** The `addRewards(...)` function is permissionless and the `distributeRewards(...)` function will schedule against an existing `lockedRewardPool` with `msg.value` of 0, so any `RewardSplitter` round, grant or donation already credited to the entity can be used as the park at no cost;
- **The owner can guarantee exclusivity.** The `updateEntityStakingBlocked(...)` function is owner callable with no timelock, so the pool can be kept shut to everyone and reopened only for the duration of the attacker's own deposit and exit, atomically within one transaction;
- **The entity is left permanently unusable.** Once `activeRewardPool` exceeds `totalShares * 1e18` the rate floors to 0, every `stake(...)` reverts with `InsufficientStakeAmount`, and no function lowers the price per share again, so any residual `lockedRewardPool` is stranded;

Two configuration preconditions gate the cross entity theft, both set by a maintainer and both intended states of the system: `bondingPeriod` must be non-zero, since at 0 the eligible branch books `costBasis = newTotalValue` and the refund can never exceed the share value, and the entity must be in `STREAM` mode. As a concrete example, three entities holding a combined $50003e18$ wei of other users' funds can be emptied by an owner who parks $1000e18$ wei and rigs their own entity at $m = 60$: after 256 cycles and a final exit the owner nets $50003.33e18$ wei, the treasury is left holding $0.67e18$ wei, and an honest staker calling the `unstake(...)` function reverts with `TransferFailed`.

Recommendation(s): Consider issuing shares in a single division, deriving them directly from the deposit, the entity's `totalShares` and its `activeRewardPool`, so that truncation falls on the last wei of the result instead of on the conversion rate. The same reasoning applies to the share issuance inside the `redelegate(...)` function and to the reverse conversion inside the `unstakeVana(...)` function.

Independently of the rounding, consider enforcing that a refund can never exceed the value of the shares being burned, so that the treasury cannot pay out more than the entity's books give up regardless of any future rounding change. Consider also whether the cost basis recorded on a stake made during the bonding period should reflect the value of the shares actually minted rather than the raw `msg.value`, which would keep the two branches of `stake(...)` consistent with each other.

Consider giving the registration stake a floor or a bond so that `totalShares` cannot be reduced to dust, which also removes the precondition for this attack and prevents the permanently unusable state. The unused `CannotRemoveRegistrationStake` error suggests this was already intended.

Finally, consider asserting as a system wide invariant that the treasury balance is at least the sum over all entities of `activeRewardPool`, `lockedRewardPool` and `accruedCommission`, and covering it in the test suite. This invariant holds on every well behaved path and is violated the moment the issue is triggered.

Status: Fixed

Update from the client:

Fixed (commit `0d83fc5`, branch `feat/staking-rewards-models`)

The attack combined three ingredients; the fix closes all three.

1. Single-division share issuance (the rounding half);

`stake` mints shares in two steps — `vanaToEntityShare` floors the rate ($\text{totalShares} \cdot 1e18 / \text{activeRewardPool}$), then multiplies by the deposit — so truncation landed on the rate, under-issuing a depositor up to $(m - 1) / m$ (50% at $m = 2$) when the rate sat just below an integer. Fixed with a single division:

1 `vanaToShares(entityId, vanaAmount) = vanaAmount * totalShares / activeRewardPool`

used in stake, redelegate's mint, and unstakeVana's eligible branch (vanaToEntityShare kept as a price view). Truncation now falls on the last wei of the result — no rig can under-issue, and the self-brick is gone.

2. Registration seed bonded + floored (the precondition). The registration stake was unbonded and fully withdrawable, letting an owner drain their entity's totalShares to dust to skew the rate. Now registerEntityStake bonds it, and _unstake reverts CannotRemoveRegistration Stake if the entity owner would drop below minRegistrationStake. On mainnet (minReg = 0.1 VANA) rigging the rate to m=2 would need 5e16 VANA parked.

3. Refund cap (already present in a commit before the meeting). _unstake refunds min(proportionalCostBasis, shareValue), so a bonding-period exit can never pay out more than the shares are worth. This is the structural backstop: treasury and activeRewardPool move in lockstep, so no entity can pull another's cash.

Net: the cross-entity drain and treasury insolvency are unreachable. We reproduced the exact attack: pre-fix it drained 413.85 VANA and reverted an honest staker's withdrawal; with the fixes the treasury stays solvent and the drain-to-dust step reverts outright. The PoC quoted the pre-fix vanaToReturn = proportionalCostBasis, i.e. a snapshot without the cap.

Verified: TreasuryDrainAttack.t.sol (attack blocked at setup; seed floored/bonded), VanaToShares.t.sol (single-division math, incl. fuzz), and a new TreasurySolvencyInvariant.t.sol asserting your recommended system-wide invariant — $\text{treasury} \geq \sum(\text{active} + \text{locked} + \text{accrued} + \text{stakerLocked})$ — held across 3.8k fuzzed operations.

Update from the client (addendum to the note above): the owner-bound floor in 0d83fc5 was found bypassable — redelegate had no floor check, and transferring entity ownership via updateEntity released it — and was replaced in 32f757f (PR #82) by a floor bound to the registrant recorded at creation, enforced on unstake, unstakeVana and redelegate-out, plus a price-per-share cap (MAX_ACTIVE_POOL_PER_SHARE = 1e12) on every mint path that protects legacy entities with no floor record. Foundry coverage: RegistrationFloorAndPrice-Cap.t.sol (15 tests); the solvency fuzzer now also drives redelegate, distributeRewards, ownership transfer and legacy (floor-less) entities (5700086).

6.2 [High] A malicious entity owner can capture or withhold the rewards the splitter has already paid to their entity's stakers

File(s): /vanaPoolEntity/VanaPoolEntityImplementation.sol, /rewardSplitter/RewardSplitterImplementation.sol

Description: The RewardSplitter contract splits a governance-funded VANA budget across VanaPool entities in proportion to the stake-seconds each entity accrued since the previous round, so an entity's share is earned by the stake its delegators committed over time. The splitter pays that share by calling the `addRewards(...)` function on the VanaPoolEntity contract, which only credits the entity's `lockedRewardPool`. For a STREAM entity, that balance reaches delegators only once a vesting schedule is installed by the `distributeRewards(...)` function, which is callable by the entity owner (or a maintainer) and accepts an unconstrained duration, including 0 for an instant release.

Rewards are therefore not credited to the stakers who earned them at the moment the splitter pays; they wait under the owner's control. Whenever they do vest, the `processRewards(...)` function skims the owner's commission at the rate in force at that moment, and `MAX_COMMISSION` is 100%.

```

1 // @audit-issue The rate applied is the one current when the amount vests, which
2 // need not be the rate that was advertised when the amount was funded.
3 uint256 commission = (toDistribute * entity.commissionRate) / MAX_COMMISSION;
4 uint256 delegatorReward = toDistribute - commission;
5
6 entity.lockedRewardPool -= toDistribute;
7 entity.activeRewardPool += delegatorReward; // to delegators via share price
8 entity.accruedCommission += commission; // operator's cut, claimable

```

The `updateEntityCommission(...)` function settles pending rewards before applying a new rate, which is intended to make rate changes forward-only. That guard does not protect funds which have not been scheduled yet: the owner raises the rate first, so the settlement has nothing to settle and the whole balance later vests under the new rate.

1. The splitter pays the entity its share X , earned by the delegators' stake-seconds. X sits in `lockedRewardPool`, vesting nothing;
2. The owner calls `updateEntityCommission(entityId, MAX_COMMISSION)`. Its settlement covers nothing, and the rate becomes 100%;
3. The owner calls `distributeRewards(entityId, X, block.timestamp, 0)` to install an instant entry. This function also settles before installing that entry, so again nothing vests at the old rate;
4. In any later block, the owner calls `claimCommission(entityId)`. It settles first, so X vests in full, commission equals X , `delegatorReward` is 0, and X is transferred from the treasury to the owner;

Steps 2 and 3 can share a single transaction. Step 4 has to be in a later block only because `processRewards(...)` returns early when no time has elapsed since the last settlement.

Once that first transaction has landed, the outcome cannot be prevented, even though `MAINTAINER_ROLE` can call the same functions the owner used. Settlement is permissionless, so any later interaction with the entity completes the capture without the owner acting again. A maintainer lowering the rate does not help, because the `updateEntityCommission(...)` function settles before applying the new rate, so their own transaction vests the balance at 100% and pays the owner. Pausing does not help either: settlement is `whenNotPaused`, so the rate cannot be changed at all while the contract is paused, and the first settlement after unpausing still applies 100%.

The owner stakes nothing, risks nothing and serves no bonding period, since `claimCommission(...)` pays `accruedCommission` directly rather than through share value. The entire budget the splitter attributed to the delegators is taken: `activeRewardPool` never grows, so their share price does not move, and `totalDistributedRewards` records nothing as distributed. The owner can then restore the advertised rate, so the `commissionRate` a delegator reads afterwards is not the rate their rewards were taken at. Entities are onboarded by a maintainer, but an entity owner is otherwise an independent operator competing for delegations.

Two weaker variants of the same root cause are available even with a bounded commission rate. The owner can stake a large amount, close the entity to others with the `updateEntityStakingBlocked(...)` function, then release X instantly so that their snapshot position captures most of it, and unstake once the bonding period passes; an early exit still returns the principal, so the downside is roughly zero. Alternatively the owner can simply never call the `distributeRewards(...)` function, leaving X parked indefinitely while the entity keeps accruing stake-seconds and drawing further budget in every later round. Delegators cannot force the release, and unstaking abandons the balance, because it belongs to the entity rather than to the stakers whose stake earned it and will accrue to whoever holds shares when it is finally released. This last variant is the one a maintainer can undo, since the `distributeRewards(...)` function is callable by `MAINTAINER_ROLE` as well, so governance can schedule a parked balance without the owner's cooperation.

Recommendation(s): Consider revisiting the assumption that an entity owner may decide the terms on which externally funded rewards reach delegators. Rewards the splitter has already attributed to an entity's stakers should not be exposed to parameters the owner can change afterwards, and the owner should not be able to choose the moment at which those rewards land relative to their own position or their own commission rate. Consider also whether an owner should be able to leave such a balance unreleased indefinitely, and what recourse delegators or governance should have when that happens.

Status: Fixed

Update from the client:

Fixed in commit 2a6c350 with 3 parts:

- Dedicated, owner-proof reward track. Splitter rewards now go through `addStakerRewards(entityId, payCommission, duration)` (`REWARD_SPLITTER_ROLE`-gated), into a separate `stakerLockedRewardPool` + `stakerRewardSchedule` kept out of `lockedRewardPool`. No owner function — `distributeRewards`, `switchToStreamModel`, `sweep`, or the `APY/STREAM` drip — can reach it, so the owner

cannot withhold, cancel, re-time, or re-rate it. It vests linearly to delegators (via share price) over an explicit duration; overlapping rounds rebase;

- Commission fixed up front. Any owner cut is skimmed atomically at the current rate when the reward is funded; the vesting portion carries no commission, so nothing can be skimmed at a later, owner-chosen moment;
- Two-phase commission changes. Increases require `proposeCommissionRate` (owner) + `approveCommissionRate` (maintainer); decreases are immediate. An owner can no longer unilaterally spike the rate before a round;

Update from Nethermind: partially fixed at [commit](#). It seems that the new `REWARD_SPLITTER_ROLE` is not granted inside `initialize` to the `RewardSplitter` contract. `addStakerRewards` will revert.

Update from the client (re: "partially fixed — `REWARD_SPLITTER_ROLE` is not granted"): Confirmed and fixed (93b28e7). The role cannot be granted in `initialize` because the splitter is deployed after the entity, so the wiring is now first-class: `VanaPoolEntity.updateRewardSplitter(address)` (`MAINTAINER_ROLE`) records the splitter (new `VanaPoolEntityStorageV2.rewardSplitter`), grants it `REWARD_SPLITTER_ROLE` and revokes the previous one, mirroring `VanaPoolTreasury.updateVanaPoolEntity`. A deploy script, `deploy/vanaStaking/rewardSplitter-deploy.ts`, deploys the splitter proxy and performs the wiring (or prints the multisig call). Tests: `RewardSplitterWiring.t.sol`, including "an unwired splitter cannot pay".

Update from Nethermind 2: An entity owner can frontrun the `approveCommissionRate(...)` call with a `proposeCommissionRate(...)` call to change the value and the `MAINTAINER` will approve an unexpected value. Consider adding a new parameter to the `approveCommissionRate(...)` function and take the rate as an input parameter and check that the input parameter matches the `pendingCommissionRate`.

Update from the client 2: Fixed in `commit 8b7a3c7`.

6.3 [Medium] A large matured position nullifies the bonding period for subsequent deposits

File(s): `/vanaPoolStaking/VanaPoolStakingImplementation.sol`

Description: The bonding period is the protocol's defensive mechanism for discouraging mercenary capital. A staker who exits before it elapses forfeits their accrued rewards into the entity's locked pool, which recycles them to the remaining stakers. When capital is added to an existing position, the contract blends the new bonding requirement with the remaining time on that position, weighted by value.

```
1 // @audit The full bonding period is divided by the combined value, so a
2 // large existing position shrinks the bond applied to new capital.
3 stakerEntity.rewardEligibilityTimestamp =
4     currentTimestamp + (stakeAmount * bondingPeriod) / newTotalValue;
```

Because the denominator is the value of the whole combined position, the bonding period applied to the incremental capital is scaled down by whatever the staker already holds. A staker holding a matured position worth 10000 VANA who then deposits 100 VANA receives a bond of 25663 seconds, roughly seven hours, instead of the configured 30 days. The larger the existing position, the closer the effective bond is to zero, and the matured position is unaffected by the operation, so the manoeuvre can be repeated indefinitely. The cost of bootstrapping the attack is serving the bonding period once.

The `redelegate(...)` function reproduces the same arithmetic, weighting the incoming amount by its carried remaining bond and the destination by its own remaining time, and then assigning rather than raising the result.

```
1 uint256 weightedTime =
2     (existingValue * existingRemaining + movedValue * remainingBond) / newTotalValue;
3 // @audit-issue The blended value is assigned, so an existing bond can be shortened.
4 to.rewardEligibilityTimestamp = currentTimestamp + weightedTime;
```

Recommendation(s): Consider revisiting the bonding model so that newly added value serves the necessary bonding period on its own terms. At a minimum, consider ensuring that neither the staking nor the redelegation path can ever move an existing eligibility deadline earlier than it already was.

Status: Acknowledged

Update from the client:

Acknowledged (WONTFIX): the value-weighted bond blend in `stake()` / `redelegate()` intentionally lowers deposit friction for stakers maintaining large already-bonded capital, weakens only an economic disincentive (no funds at risk, no accounting invariant broken, matured gains preserved by c583cc7), and the clean per-tranche fix costs storage/complexity or perpetually re-locks honest large stakers — so we keep it as-is.

6.4 [Medium] Accrued commission can never be withdrawn from the treasury

File(s): /vanaPoolEntity/VanaPoolEntityImplementation.sol, /vanaPoolTreasury/VanaPoolTreasuryImplementation.sol

Description: Each entity may charge a commission, which is an operator cut taken from every reward distribution. The processRewards(...) function from the VanaPoolEntity contract takes that cut into the accruedCommission field, and the entity owner later withdraws it by calling the claimCommission(...) function, which instructs the treasury to pay out. All VANA in the system is custodied by the VanaPoolTreasury contract, and moving funds out of it requires the caller to hold DEFAULT_ADMIN_ROLE.

```

1 // @audit-issue The caller of this function is the VanaPoolEntity contract.
2 bool success = vanaPoolStaking.vanaPoolTreasury().transferVana(
3     payable(entity.ownerAddress),
4     amount
5 );
6 if (!success) {
7     revert TransferFailed();
8 }

```

The treasury grants that role to two addresses when it is initialized, and the updateVanaPool(...) function only rotates it between staking contracts. The VanaPoolEntity contract is never granted the role.

```

1 function initialize(address ownerAddress, address vanaPoolAddress) external initializer {
2     // ...
3     _grantRole(DEFAULT_ADMIN_ROLE, ownerAddress);
4     // @audit-issue Only the owner and the staking contract are granted the role.
5     _grantRole(DEFAULT_ADMIN_ROLE, vanaPoolAddress);
6 }

```

Every reward distribution therefore reduces the delegators' share by the commission, while every attempt to claim that commission reverts due to the treasury's access control check.

Recommendation(s): Consider introducing a dedicated, narrowly scoped spending role on the VanaPoolTreasury contract that gates only the transfer functionality, and granting it to both the staking and the entity contracts as part of the upgrade procedure. Granting DEFAULT_ADMIN_ROLE to the entity contract would also resolve the revert, but it is strongly discouraged, because that role additionally authorizes upgrades and pausing of the treasury that custodies all staker principal.

Status: Fixed

Update from the client: Fixed (commit 6b7b2d6)

Confirmed real: claimCommission has the entity contract call treasury.transferVana, but transferVana required DEFAULT_ADMIN_ROLE and the entity was never granted it, so every claim reverted. (On this branch the same would have broken sweepUnallocatedRewards, which also pays out through the entity.)

Fix — a narrow SPENDER_ROLE gating only transferVana:

initialize grants SPENDER_ROLE to the staking contract and DEFAULT_ADMIN only to the owner — staking no longer holds admin over the treasury. updateVanaPool rotates SPENDER_ROLE (not DEFAULT_ADMIN) between staking contracts. New updateVanaPoolEntity(address) (admin-gated) records the entity and grants it SPENDER_ROLE, rotating it off any previous entity — first-class wiring so the entity's spend right can't be silently forgotten. Backed by one appended slot (VanaPoolTreasuryStorageV2).

6.5 [Medium] Redelegating into a matured position makes already earned rewards forfeitable

File(s): /vanaPoolStaking/VanaPoolStakingImplementation.sol

Description: Stakers serve a bonding period, during which an early exit returns only the principal and forfeits the accrued rewards. Once that period has elapsed the position is matured and its rewards are withdrawable. The `redelegate(...)` function from the `VanaPoolStaking` contract lets a staker move a position between entities, and its documentation states that the full share value, principal plus accrued rewards, travels with the staker and that there is no forfeiture.

When a staker adds to an already matured position through the `stake(...)` function, the contract first records the matured gain into `vestedRewards` and resets the cost basis to the current value, so that the gain can no longer be lost before it re-bonds the position.

```

1  if (stakerEntity.rewardEligibilityTimestamp <= currentTimestamp) {
2      if (stakerEntity.shares > 0) {
3          uint256 oldValue = (stakerEntity.shares * shareToVana) / 1e18;
4          if (oldValue > stakerEntity.costBasis) {
5              stakerEntity.vestedRewards += oldValue - stakerEntity.costBasis;
6          }
7      }
8      // @audit The cost basis absorbs the matured gain before the bond is reset.
9      stakerEntity.costBasis = newTotalValue;
10     // ...
11 }

```

The destination side of the redelegation performs no such record. It adds only the principal to the cost basis and then writes a new, future eligibility timestamp over the destination position.

```

1  // @audit-issue The matured gain is not crystallised before the position is re-bonded.
2  to.costBasis += movedCostBasis;
3  to.vestedRewards += movedVested;
4  to.shares += sharesIssued;

```

A staker who holds a matured position in one entity and a bonded position in another, and who moves the second into the first puts their previously safe rewards back at risk. If they exit before the newly imposed bond elapses, the unstaking logic takes the bonding branch, returns only the cost basis, and routes the difference back into the entity's locked pool for other stakers and the entity owner's commission.

Recommendation(s): Consider mirroring the logic that the staking path already performs, so that a matured destination position has its unrealised gain converted into cost basis before any new bonding deadline is applied to it. Consider also reviewing whether the documented promise that no forfeiture occurs during redelegation holds for every combination of source and destination maturity.

Status: Fixed

Update from the client:

Pushed — c583cc7. Here's the auditor summary:

[Medium] Redelegating into a matured position makes earned rewards forfeitable — Fixed (commit c583cc7)

Root cause confirmed: stake into a matured position crystallizes the earned gain into cost basis before re-bonding (`costBasis = newTotalValue`), so a later bonding-period exit still returns it. `redelegate`'s destination side did not — it added only the moved principal (`to.costBasis += movedCostBasis`) and wrote a new future eligibility over the merged position. A staker holding a matured position in A and a bonded position in B who redelegated B→A had A re-bonded with its gain not crystallized; exiting A before the new bond took the bonding branch and forfeited that gain — breaking the documented "full value travels, no forfeiture."

Fix: on the destination, before applying the new bond, crystallize to's gain when it is matured (mirrors stake's eligible branch):

```
if (existingRemaining == 0 to.shares > 0 existingValue > to.costBasis) to.vestedRewards += existingValue - to.costBasis; to.costBasis = existingValue;
```

`redelegate` is now a proper superset of the manual path — it preserves B's rewards (which the manual path forfeits on B's bonded exit) and protects A's matured gain.

Update from Nethermind: The updated logic only checks if the recipient has a matured position, but it doesn't check if `from`'s position is matured. This means that if a matured position is redelegated, the rewards become subject to the bounding period again. Consider using a similar approach to the one used for realizing the rewards for the `to`'s position.

Update from the client: Confirmed and fixed (787e074). `redelegate` now crystallizes a matured source's unrealized gain into the carried cost basis and `vestedRewards` before the merge (the same pattern `stake()` applies when topping up a matured position), so a bond imposed by the destination cannot make it forfeitable; the source is debited by the proportional originals. Test `test_scenario2_redelegateMaturedAintoBondedB`: a 72.97 VANA matured gain moved into a bonded position and exited early is paid in full (before: forfeited).

6.6 [Medium] Reward distribution weight can be increased by settling rewards early

File(s): /rewardSplitter/RewardSplitterImplementation.sol, /vanaPoolEntity/VanaPoolEntityImplementation.sol

Description: A governance-funded budget is split across entities by the `distribute(...)` function from the `RewardSplitter` contract, in proportion to the stake-seconds each entity accrued since the previous round. Stake-seconds are the integral of an entity's `activeRewardPool` over time, and the splitter's documentation states that the design prevents a third party from skewing the split.

The quantity being integrated is also the quantity that reward settlement increases. The `processRewards(...)` function from the `VanaPool` Entity contract moves value from the locked pool into the active pool, and it carries no access control at all.

```

1 // @audit-issue This function is permissionless and raises the value that the
2 // stake-seconds metric integrates over time.
3 function processRewards(uint256 entityId) public override whenNotPaused {
4     // ...
5     entity.lockedRewardPool -= toDistribute;
6     entity.activeRewardPool += delegatorReward;
7     // ...
8 }

```

An entity that settles its pending rewards at the start of a distribution round therefore carries an elevated pool for the whole round and accrues more stake-seconds than an otherwise identical entity that settles at the end, even though both hold the same delegated stake and receive the same rewards. Anyone may trigger the settlement for any entity, so the advantage costs only gas and can be taken every round.

The effect is not marginal. A proof of concept with two entities that are identical in stake, locked rewards and timing, differing only in when their pending rewards are realised, splits a 100 ether budget 66.655562958027981345 ether to 33.344437041972018654 ether, a two-to-one division of a budget that should have been shared equally.

Recommendation(s): Consider weighting the distribution by delegated principal rather than by the reward-inclusive pool, for example by maintaining a separate accumulator that is updated only when stake enters or leaves an entity. Consider also reviewing the documented claim that the split cannot be skewed, since it currently addresses only attempts to reduce another entity's weight and not attempts to raise one's own.

Status: Fixed

Update from the client:

Fixed in d7d52ff: the cross-entity split now weights entities by committed principal (`stakedPrincipal`, moved only on real stake/unstake/re-delegate) instead of the time-integral of `activeRewardPool`, and `processRewards` no longer checkpoints the accumulator — so vesting rewards early can no longer inflate an entity's weight (verified by a two-identical-entities test where 20 days of aggressive settlement still yields a 50/50 split; 180/180).

Update from Nethermind: the `_checkpointPrincipalSeconds(...)` is only called in `updateEntityPool(...)` and it is supposed to only include the principal amount and not rewards. After the pool is upgraded, but before a `stake(...)` or `unstake(...)` function call happens, if a reward is distributed by the entity owner, then `stakedPrincipal` will be set equal to `activeRewardPool` which will also contain the newly distributed rewards. This can only happen in legacy pools though.

Update from the client: Confirmed and fixed (4d3fde0). A pre-upgrade entity's `stakedPrincipal` is now seeded from `min(totalShares, activeRewardPool)` rather than `activeRewardPool`: rewards never mint shares, so nothing settled into the pool between the upgrade and the first stake/unstake can enter the seed, and the share supply is a lower bound on principal (because a deposit of v VANA at price p mints v/p shares, the share supply is a lower bound on principal; it can only under-weight a legacy pool). A permissionless `checkpointPrincipal(entityId)` lets the upgrade procedure seed legacy entities immediately, closing the window itself. Tests: 400 shares against a 500 pool seed 400 (before: 500); two entities with equal share supply and $1000\times$ different pool values seed identically.

6.7 [Medium] Rounding dust left by the final unstake disables an entity

File(s): /vanaPoolEntity/VanaPoolEntityImplementation.sol

Description: Staking positions in an entity are accounted as shares. The `entityShareToVana(...)` and `vanaToEntityShare(...)` functions from the `VanaPoolEntity` contract convert between shares and VANA and are intended to be reciprocals of one another. Both must also address the case of an empty entity, where the next depositor is expected to bootstrap the pool at a 1:1 price.

The two functions treat this case differently. The first guards on the share supply, whereas the second guards on the size of the pool.

```

1  function entityShareToVana(uint256 entityId) external view override returns (uint256) {
2      Entity storage entity = _entities[entityId];
3
4      return entity.totalShares > 0
5          ? (entity.activeRewardPool * 1e18) / entity.totalShares
6          : 1e18;
7  }
```

The reciprocal branches on `activeRewardPool` instead, so for an entity that holds VANA but has no shares outstanding it returns 0 rather than the bootstrap price.

```

1  function vanaToEntityShare(uint256 entityId) external view override returns (uint256) {
2      Entity storage entity = _entities[entityId];
3
4      // @audit-issue This branches on the pool size rather than the share supply.
5      return entity.activeRewardPool > 0
6          ? (entity.totalShares * 1e18) / entity.activeRewardPool
7          : 1e18;
8  }
```

This state can be reached when the last holder exits. The VANA returned to them is computed through two successive floor divisions, so whenever the share supply does not divide the pool exactly, a remainder is left behind while the share count reaches zero. With an `activeRewardPool` of `4e18` against a `totalShares` of `3e18`, the final exit returns `3999999999999999999` and strands 1 wei.

From that point on the entity is unusable. Every call to the staking function computes zero shares issued and reverts, and every redelegation into the entity reverts on its slippage check, so the share supply can never become non-zero again. The `activeRewardPool` is decreased through paths that now revert, so the entity remains listed as active while its `lockedRewardPool`, including every payout the reward splitter routes to it, is unrecoverable.

Recommendation(s): Consider making both conversion functions agree on a single definition of the empty entity, keyed on the share supply, so that an entity holding residual VANA with no shares outstanding still quotes a bootstrap price. Additionally, consider sweeping any residual `activeRewardPool` back into the locked pool at the moment the share supply reaches zero, so that the dust does not accumulate against an entity with no holders.

Status: Fixed

Update from the client: Update from the client: Fixed (`f3c72e3` zero-share price floor; `0d83fc5` single-division `vanaToShares`). Both conversions key the empty entity on the share supply and quote the 1:1 bootstrap price. Dedicated test added in `770b9ae` (`DustBootstrap.t.sol`): reproduces the stranded-dust state on a legacy entity, then asserts a stake mints 1:1, a redelegation in succeeds, and locked rewards are not stranded, with treasury solvency throughout.

Update from Nethermind: fixed at [commit](#).

6.8 [Medium] The executeBuyAndBurn(...) function sends tokens to a configurable sink address instead of destroying them

File(s): ../RewardSplitterImplementation.sol

Description: The RewardSplitterImplementation contract includes a buy-and-burn mechanism that sets aside a portion of the distributed tokens to be burned later. The executeBuyAndBurn() function is responsible for executing this burn by transferring the accrued native tokens. However, the function sends these funds to a configurable sink address rather than address(0).

The protocol's burn mechanism is intended to reduce the totalSupply of tokens in order to increase the price per token. Because the current logic transfers the tokens to a "sink" address instead of the zero address, the tokens are never actually destroyed. Consequently, the totalSupply does not change, meaning the feature is broken and will not achieve its desired economic goal.

```
1  function executeBuyAndBurn() external nonReentrant {
2      uint256 amount = pendingBurn;
3      if (amount == 0) {
4          return;
5      }
6      address sink = buyAndBurnAddress;
7      if (sink == address(0)) {
8          revert InvalidAddress();
9      }
10     pendingBurn = 0;
11     // @audit-issue Sending funds to a sink address does not reduce the token total supply.
12     (bool burned, ) = sink.call{value: amount}("");
13     if (!burned) {
14         revert TransferFailed();
15     }
16     emit BuyAndBurnExecuted(sink, amount);
17 }
18
```

Recommendation(s): Consider revisiting the burn logic to ensure that the accrued tokens are sent to the zero address or properly removed from circulation.

Status: Fixed

Update from the client: Fixed. sink was removed and the address(0) is used to burn.

Update from Nethermind: fixed at [commit](#).

6.9 [Medium] The entity registration stake never serves a bonding period

File(s): /vanaPoolStaking/VanaPoolStakingImplementation.sol

Description: When a new entity is created, its owner must deposit a registration stake, which acts as the operator's collateral and is recorded as an ordinary staking position through the `registerEntityStake(...)` function from the `VanaPoolStaking` contract. Every other staking position is opened through the `stake(...)` function, which writes a share count, a cost basis and a bonding deadline.

The registration path writes only the first two fields.

```
1 // Register shares for the owner
2 StakerEntity storage stakerEntity = _stakers[ownerAddress].entities[entityId];
3 stakerEntity.shares = registrationStake;
4 // @audit-issue The rewardEligibilityTimestamp is never written and defaults to 0.
5 stakerEntity.costBasis = registrationStake;
6
7 _addStaker(ownerAddress);
```

Every reward in the unstaking logic compares the current time against `rewardEligibilityTimestamp` using a `>=` comparison, so the default value of `0` is read as a deadline that elapsed. The entity owner is therefore eligible for rewards on their collateral, they may withdraw it at any moment, receive the full share value including all accrued rewards, and forfeit nothing, while every ordinary staker in the same entity who deposited at the same time must serve the full bonding period or forfeit.

Collateral was intended to be locked and the evidence comes from the contract itself, which declares a dedicated error for this case, but it is never used.

```
1 // @audit-issue This error is declared but is never used anywhere in the codebase.
2 error CannotRemoveRegistrationStake();
```

Beyond the unequal treatment between an entity creator and a regular staker, removing the collateral leaves an entity that is still listed as active and eligible to receive reward splitter payouts, with no operator stake behind it. It also produces the empty entity state that the reward stream capture issue depends on.

Recommendation(s): Consider assigning a bonding deadline to the registration position when it is created, so the operator is subject to the same forfeiture rules as the delegators they attract. Consider additionally enforcing the already declared restriction so that the collateral cannot be withdrawn while the entity remains active.

Status: Fixed

Update from the client: Fixed. Bonding period is attached to the registration stake. Also, the owner cannot unstake and leave the stake in the pool less than `minRegistrationStake`.

Update from the client (addendum): the seed is bonded in `registerEntityStake(0d83fc5)` and the non-removable floor is recorded per registrant in `VanaPoolStakingStorageV3(32f757f)`, so neither `unstake`, `unstakeVana`, `redelegate-out` nor an ownership transfer can take `totalShares` below it.

6.10 [Low] A one wei deposit can capture an entire elapsed reward stream

File(s): /vanaPoolEntity/VanaPoolEntityImplementation.sol

Description: Entities may vest rewards under a stream model, in which a scheduled amount is released linearly over a time period. The `_vestStream(...)` function from the `VanaPoolEntity` contract computes how much of the active schedule has matured since its last update and advances a checkpoint so the same interval is never paid twice.

When an entity has no shares, there is nobody to receive a distribution, so the function returns early. Deliberately, and as documented in the comment, it does so without advancing the checkpoint, in order to preserve the elapsed interval for whoever stakes next.

```
1 function _vestStream(  
2     RewardSchedule storage schedule,  
3     uint256 totalShares  
4 ) internal returns (uint256 toVest) {  
5     // @audit-issue The checkpoint is not advanced, so the whole elapsed interval  
6     // is preserved and paid to whoever holds shares next.  
7     if (schedule.scheduledValue == 0 || totalShares == 0) {  
8         return 0;  
9     }  
10    // ...  
11 }
```

The preserved value is not reserved for the stakers it was intended for; it is paid to whatever share supply exists at the next settlement. Because the entity is empty, the share price falls back to 1:1, so a deposit of 1 wei mints one share and makes the depositor the sole owner of the entity. Settling the rewards then moves the entire preserved amount into a pool backed by that single share.

Crucially, the guard that protects the APY model does not apply here. The staking function settles rewards before it mints shares, which for an APY entity credits any backlog to the preexisting holders. For a stream entity that settlement hits the early return above and credits nothing, so the new depositor's share is minted first and the stream vests to them afterwards.

Recommendation(s): Consider advancing the checkpoint when an entity has no shares outstanding, so that no vesting accrues to an empty pool, and returning the elapsed portion to the entity's residue instead. If preserving the interval is a deliberate product requirement, consider gating the preserved amount so that it can only be released to a share supply that existed while it accrued.

Status: Fixed

Update from the client: Fixed (d8e459d). `_vestStream` now advances the watermark (and promotes ended entries) even when `totalShares == 0`; the elapsed portion vests to nobody and stays in `lockedRewardPool` as re-schedulable residue rather than being preserved for the next depositor. Reproduced your PoC: after 50 days empty, a 1 wei deposit was worth 50 ether + 1 wei; after the fix, dust, with the 50 ether retained and fully re-schedulable by the owner. Tests: `StreamEmptyPoolCapture.t.sol`, `VestStream.t.sol`.

6.11 [Low] Switching reward models leaves a stale stream schedule installed

File(s): /vanaPoolEntity/VanaPoolEntityImplementation.sol

Description: An entity vests rewards either through a continuously compounding APY drip or through a linear stream, and a maintainer may move an entity between the two models in either direction using the `updateEntityRewardModel(...)` function from the `VanaPoolEntity` contract. The two models keep separate clocks: the APY drip advances `lastUpdateTimestamp`, while the stream advances its own checkpoint inside the schedule.

The dedicated function that migrates an entity into the stream model clears any previously installed schedule, and its comment explains that this is to prevent a stale entry from surviving.

```
1 processRewards(entityId);
2 entity.rewardModel = RewardModel.STREAM;
3 // @audit This sibling function clears the schedule before installing a new one.
4 delete entity.rewardSchedule;
```

The general model switch performs no equivalent clearing. It settles the outgoing model and then flips the flag, leaving the schedule and its checkpoint untouched.

```
1 processRewards(entityId);
2 // @audit-issue The reward schedule and its checkpoint survive the switch to APY.
3 entity.rewardModel = model;
```

While the entity sits in APY mode, the drip consumes the same `lockedRewardPool` that still escrows the untouched stream, and the escrow invariant is only ever checked when a new distribution is scheduled. Switching back reactivates a schedule whose checkpoint predates the entire APY phase and whose window may have elapsed in the meantime, so the next settlement releases the remaining escrow as a single lump rather than linearly.

A proof of concept installs a 100 ether stream over 100 days, an intended rate of 1 ether per day, switches to APY after one day, runs the drip for 200 days, and switches back. The next settlement releases 65.510030697081739698 ether in a single block and empties the locked pool entirely. Any shortfall between what the resurrected schedule claims and what remains in the locked pool is silently absorbed by a clamp that runs after the checkpoint has already moved, so the difference can never be vested afterwards.

Recommendation(s): Consider making the general model switch symmetric with the dedicated one, so that leaving the stream model clears or anchors the schedule and its checkpoint. Consider also re-asserting the escrow invariant whenever an entity reenters the stream model, so that a schedule can never be reactivated against a pool that no longer backs it.

Status: Fixed

Update from the client: Fixed (d898734). `updateEntityRewardModel` was removed entirely; the only switch is the one-way switch-`ToStreamModel`, which deletes any existing schedule before installing a new one.

Update from Nethermind: fixed at [commit](#). The function has been removed. The `switchToStreamModel` function should be used instead.

6.12 [Low] The VANA denominated unstake disables the minimum output check

File(s): /vanaPoolStaking/VanaPoolStakingImplementation.sol

Description: The VanaPoolStaking contract offers two ways to withdraw. The share denominated unstake(...) function forwards a caller-supplied minimum VANA amount, which the internal logic enforces against the amount actually returned. The VANA denominated unstakeVana(...) function converts the requested amount into shares and then delegates to the same internal logic.

It passes a hard-coded zero in place of that minimum, so the guard its sibling offers is disabled on this path.

```
1 // @audit-issue The minimum VANA output is hard coded to zero on this path.
2 _unstake(staker, entityId, shareAmount, 0, true);
```

The only protection available to the caller bounds the shares burned rather than the VANA received, and it is applied after a silent clamp, so a request larger than the position is quietly reduced to the whole position instead of reverting.

```
1 if (shareAmount > stakerEntity.shares) {
2     // @audit-issue An oversized request is silently reduced instead of reverting.
3     shareAmount = stakerEntity.shares;
4 }
5
6 if (shareAmountMax > 0 && shareAmount > shareAmountMax) {
7     revert InvalidSlippage();
8 }
```

A caller who passes the documented skip value for the share bound therefore executes with no protection on either side. The realistic exposure is modest, because the share price is monotonically non-decreasing and both conversions occur in the same transaction, but the asymmetry between the two withdrawal functions is unlikely to be intentional.

Recommendation(s): Consider offering the same value denominated protection on both withdrawal paths, so that a caller specifying an amount in VANA can bound what they receive. Consider also reverting rather than silently clamping when the requested amount exceeds the position, so that callers are not served a materially different operation from the one they requested.

Status: Fixed

Update from the client: Fixed (77ab6a5), per your recommendation. unstakeVana takes a fourth parameter, vanaAmountMin, forwarded to _unstake exactly as unstake() does, and reverts with InvalidAmount on a request larger than the position instead of silently clamping. unstakeVana is not in the mainnet ABI, so the signature change affects no integrator. We deliberately did not make vanaAmount itself the implicit floor: shares are floored from the request and valued by a floored rate, so the payout lands 1–2 wei under the request on any non-1:1 price (measured 365/365 sampled requests), and an exact floor reverted essentially every call; the docstring directs callers to a floor with dust tolerance. Tests: UnstakeVanaMinOut.t.sol.

6.13 [Low] The computed bonding deadline is bounded by neither the configured period nor its maximum

File(s): /vanaPoolStaking/VanaPoolStakingImplementation.sol

Description: The protocol advertises a single bonding period, and the `updateBondingPeriod(...)` function from the `VanaPoolStaking` contract validates that the configured value never exceeds a hard ceiling.

```

1  function updateBondingPeriod(uint256 newBondingPeriod) external onlyRole(MAINTAINER_ROLE) {
2      // @audit The ceiling is validated here, on the CONFIGURED value only.
3      if (newBondingPeriod > MAX_BONDING_PERIOD) {
4          revert InvalidBondingPeriod();
5      }
6      bondingPeriod = newBondingPeriod;
7      // ...
8  }
```

The deadline actually written to a position is not that value but a quotient, and nothing validates the result against either the configured period or the ceiling.

```

1  // @audit-issue Neither branch bounds the result by bondingPeriod or MAX_BONDING_PERIOD.
2  stakerEntity.rewardEligibilityTimestamp =
3      currentTimestamp + (stakeAmount * bondingPeriod) / newTotalValue;
4  // ...
5  uint256 weightedTime =
6      (oldValue * remainingTime + stakeAmount * bondingPeriod) / newTotalValue;
```

This produces two behaviours that the advertised period does not describe. First, `newTotalValue` is derived through successive floor divisions, so for a very small deposit it lands below `stakeAmount` and the quotient overshoots. Measured against a share price of 1.1188 and a configured period of 432000 seconds, the smallest deposit that mints a single share receives a bond of 864000 seconds, exactly 2 times the configured value; the factor is $1 + 1/n$ for n shares minted and has decayed to nothing by a deposit of $1e6$ wei. With the period set to `MAX_BONDING_PERIOD`, the same deposit yields 63072000 seconds against a stated ceiling of 31536000. No economically meaningful deposit is affected, but the ceiling is not the invariant it appears to be.

Second, in the opposite direction, a top-up made part-way through a bond discards most of the time already served on the entire position, because the blend is recomputed over the combined value. With a period of 30 days, a position ten days into its bond has 1728000 seconds remaining; adding ten times the position returns the remaining time to 2513454 seconds, close to a full restart. This is the intended anti-wash behaviour rather than a defect, but it is not documented for users and it is the same formula that, in the matured-anchor case reported separately, collapses the bond toward zero instead.

Recommendation(s): Consider bounding the computed deadline so that a position can never be bonded for longer than the configured period, and re-asserting the ceiling on the result rather than only on the configuration. Consider also documenting that adding to a position recomputes the deadline for the whole position, since both the overshoot and the near-restart surprise users who reason about the advertised period alone.

Status: Fixed

Update from the client: Fixed (0c30045). The stake bonding-branch blend and the redelegate destination blend are capped at `bondingPeriod`, mirroring the existing partial-unstake and redelegate-source clamps. Reproduced 28.8-day and 29.0-day deadlines written against a 7-day period after a governance reduction; both now 7 days. Upper bound only, so the value-weighted behaviour accepted under the separate Medium is unchanged. Tests: `BondingDeadlineBound.t.sol`.

Update from Nethermind: the `if (stakerEntity.rewardEligibilityTimestamp <= currentTimestamp)` branch doesn't bound `rewardEligibilityTimestamp` to the `bondingPeriod`. The same capping mechanism should be applied to this branch as well.

Update from the client: Confirmed and fixed (602e9e8). `stake()`'s eligible branch now caps `stakeAmount * bondingPeriod / newTotalValue` at `bondingPeriod`, consistent with the other three deadline writes. The overshoot arises because `newTotalValue` is the minted shares' floored value: at a high price per share a deposit worth 1.9 shares mints one, giving a ratio of 1.9. Test `test_eligibleBranchDeadlines-CappedAtBondingPeriod`: such a deposit into a $5e11$ wei/share pool now receives exactly `bondingPeriod` (before: 1.9x).

6.14 [Low] The maximum unstake view simulates only the APY reward model

File(s): `contracts/vanaStaking/vanaPoolStaking/VanaPoolStakingImplementation.sol`

Description: The `getMaxUnstakeAmount(...)` function from the `VanaPoolStaking` contract tells an integrator how much a staker can withdraw right now. Its documentation states that it simulates reward settlement in order to quote accurate share prices, and front ends are expected to size withdrawals from the values it returns.

The simulation reproduces only one of the two reward models, and omits the operator commission entirely.

```
1 // @audit-issue The APY formula is applied unconditionally, with no check of the
2 // @audit-issue entity's reward model and no deduction for commission.
3 uint256 toDistribute =
4     vanaPoolEntity.calculateYield(simulatedActiveRewardPool, entityInfo.maxAPY, timeElapsed);
5 if (toDistribute > entityInfo.lockedRewardPool) {
6     toDistribute = entityInfo.lockedRewardPool;
7 }
8 simulatedActiveRewardPool += toDistribute;
```

Actual settlement branches on the entity's reward model and credits only the delegator portion of the amount released. For an entity using the stream model the view invents a drip that will never occur, while ignoring the linear vesting that will, so the quote is wrong in both directions. For any entity with a non-zero commission the simulated pool is overstated by the commission share. The structure of the data the view reads makes a correct answer impossible, since the entity information struct carries neither the reward model nor the commission rate.

There is no direct impact on funds, as the function does not modify state and the returned share count is clamped to the staker's actual holdings. The consequence falls on integrators, whose withdrawals either revert against a slippage bound or return less than quoted. The view additionally reports the treasury balance as available headroom for a single entity, when that balance is shared across all entities.

Recommendation(s): Consider having the view consult the entity's reward model, schedule and commission rate so that it reproduces what settlement will actually credit. A more robust alternative is to expose a preview of reward settlement on the `VanaPoolEntity` contract and have the view call it, so that the quoting and executing paths cannot drift apart as the reward logic evolves.

Status: Fixed

Update from the client: Fixed (4197369), taking your stronger recommendation. `VanaPoolEntity.previewActiveRewardPool()` mirrors `processRewards` read-only (actual reward model, `lockedRewardPool` cap, commission skim, splitter track, pending queued promotion), and `getMaxUnstakeAmount` quotes from it. The vesting arithmetic was extracted into a single `_streamDelta` helper shared by the executing and quoting paths, so they cannot drift. Reproduced 100.13 vs 107.13 ether on a `STREAM` entity before; exact equality after, fuzzed over 1–400 days. The view also quotes the whole position exactly when it is the binding constraint. Tests: `MaxUnstakePreview.t.sol`.

6.15 [Low] Unclamped decrement of distributed rewards blocks early unstaking

File(s): [/vanaPoolEntity/VanaPoolEntityImplementation.sol](#)

Description: When a staker exits before their bonding period elapses, the rewards they forfeit are returned to the entity so they can be redistributed. The `returnForfeitedRewards(...)` function from the `VanaPoolEntity` contract performs that return and also decrements the entity's cumulative `totalDistributedRewards` counter, since the forfeited amount is no longer considered distributed.

```
1 entity.lockedRewardPool += amount;
2 // @audit-issue This subtraction has no lower bound and reverts on underflow.
3 entity.totalDistributedRewards -= amount;
```

The sibling function that moves the same amount between entities during a redelegation explicitly guards the identical subtraction, and its comment states precisely why.

```
1 // @audit The author clamps the same counter here, but not in the function above.
2 uint256 debit = amount < from.totalDistributedRewards ? amount : from.totalDistributedRewards;
3 from.totalDistributedRewards -= debit;
```

The counter only ever accumulates the delegator portion of each distribution, while share price appreciation caused by floor rounding raises the value of existing positions without being credited to it. The difference between the sum of forfeitable gains and the counter is therefore thin, and a small residual is enough to invert the comparison. Because the minimum stake amount is not enforced, a deposit of 2 wei is sufficient to inject such a residual.

Once the counter is short, the affected staker's early exit reverts with an arithmetic panic inside the reward return, and their principal is inaccessible. The DOS is temporary rather than permanent: once the bonding period elapses, the matured branch of the unstaking logic runs, nothing is forfeited, and the reward return is never reached, so the position can always be withdrawn eventually.

Recommendation(s): Consider applying the same clamp that the redelegation path already uses, so that the counter returns zero instead of reverting. Consider also reviewing whether `totalDistributedRewards` should track rounding-induced appreciation at all, so that the two quantities being compared cannot drift apart.

Status: Fixed

Update from the client: Update from the client: Fixed (f3c72e3). `returnForfeitedRewards` now clamps `totalDistributedRewards` to zero instead of underflowing, matching the redelegation path.

6.16 [Info] Redelegating can revert when the cost basis exceeds the share value

File(s): [contracts/vanaStaking/vanaPoolStaking/VanaPoolStakingImplementation.sol](#)

Description: When a position is redelegated, the `redelegate(...)` function from the `VanaPoolStaking` contract transfers the reward portion of the moved value to the destination entity's accounting ledger. That reward portion is computed as the difference between the moved value and the moved principal.

```
1 // @audit-issue This subtraction is unguarded and reverts when the cost basis
2 // @audit-issue exceeds the current value of the moved shares.
3 vanaPoolEntity.redelegateDistributedRewards(
4     fromEntityId,
5     toEntityId,
6     movedValue - movedCostBasis
7 );
```

The two operands are not derived symmetrically. Cost basis is credited at full precision when a position is opened, whereas the shares backing it are minted through two successive floor divisions. A position can therefore be worth marginally less than its recorded cost basis, and the subtraction underflows. In a worked example a zero-gain position of 5000 ether leaves a deficit of 2692 wei, which is sufficient to make the redelegation revert with an arithmetic panic.

The impact is limited. Funds are not at risk and are not locked, because the ordinary unstaking path does not perform this subtraction and remains available. The condition also resolves itself as soon as reward accrual lifts the share value above the cost basis, so the affected position is only temporarily unable to be moved.

Recommendation(s): Consider clamping the transferred reward portion at zero so that a position worth marginally less than its cost basis does not make redelegation revert. Consider also reviewing whether cost basis should be recorded at the value actually backed by the minted shares, so that the two quantities cannot diverge in the first place.

Status: Fixed

Update from the client: Fixed (224d5a1, merged in 32f757f). The transferred reward portion is clamped at zero (`movedValue > movedCostBasis ? ... : 0`), and bonded top-ups are booked at minted-share value so the two quantities no longer diverge. Test: `test_redelegateDoesNotRevertWhenRoundingLeftPrincipalAWeiAboveValue`.

6.17 [Info] The `_vestStream(...)` function casts the timestamp to a variable that will overflow in the future

File(s): `/vanaPoolEntity/VanaPoolEntityImplementation.sol`

Description: The `_vestStream(...)` function from the `VanaPoolEntityImplementation` contract is used to compute how much a schedule has vested. The function converts the `block.timestamp` value to a `uint32` and stores it in the `schedule.lastUpdate` variable. Since this is a Unix timestamp expressed in seconds, the maximum value that a `uint32` can hold corresponds to approximately 136 years. Given that the Unix epoch started in 1970 and the current year is 2026, 56 years have already passed. This means the contract will behave as expected for about 80 more years until the `uint32` cast silently overflows and breaks the reward vesting logic.

```
1 function _vestStream(RewardSchedule storage schedule, uint256 totalShares) internal returns (uint256 toVest) {  
2     // ...  
3     // @audit Casting block.timestamp to uint32 will overflow in approximately 80 years.  
4     schedule.lastUpdate = uint32(block.timestamp);  
5 }  
6
```

Recommendation(s): Consider updating the timestamp storage type to a larger data type to prevent potential overflow in the future.

Status: Fixed

Update from the client:

Update from Nethermind: fixed at [commit](#)

6.18 [Info] The committedRewards(...) view can report a stale value before rewards are settled

File(s): `contracts/vanaStaking/vanaPoolEntity/VanaPoolEntityImplementation.sol`

Description: An entity in the stream reward model escrows its future rewards in `lockedRewardPool`, and the `_committedRewards(...)` function reports how much of that escrow the active schedule entry and its single queued follow on entry still owe. The value is used internally to guarantee that the locked pool always backs everything the schedule promises, and it is also exposed to integrators through the `committedRewards(...)` view.

```
1 function committedRewards(uint256 entityId) external view override returns (uint256) {
2     return _committedRewards(_entities[entityId].rewardSchedule);
3 }
```

The `_committedRewards(...)` function decides how much of the active entry is still outstanding purely from the current time, and it always adds the queued entry at its full `nextScheduledValue`.

```
1 } else if (block.timestamp >= end) {
2     // @audit-issue Reports 0 as soon as the entry's end time passes, which is
3     // only correct once processRewards has vested and promoted the entry.
4     activeRemaining = 0; // fully vested
5 }
6 // ...
7 // @audit The queued entry is always added in full, even after its own vesting
8 // window has started.
9 return activeRemaining + schedule.nextScheduledValue;
```

These assumptions hold only right after the `processRewards(...)` function has settled the schedule and promoted an ended entry. When the view is read on an unsettled schedule, that is, an active entry that has ended but has not yet been promoted, or a queued entry whose vesting window has already started, the figure it returns does not match what the schedule actually still owes. The ended active entry is dropped to 0 while its funds still sit unmoved in `lockedRewardPool`, and the queued entry is counted at its full value even though part of it would already have vested. In practice this over reports the outstanding amount rather than under reporting it.

There is no impact on funds or on the escrow guarantee. Every state changing consumer of `_committedRewards(...)` calls `processRewards(...)` first, so the internal check that the locked pool covers the committed amount is always evaluated against a freshly settled schedule.

```
1 processRewards(entityId);
2 // ...
3 if (entity.lockedRewardPool < _committedRewards(entity.rewardSchedule)) {
4     revert InsufficientRewardFunds();
5 }
```

The consequence is therefore limited to the public `committedRewards(...)` view: an integrator that reads it without first triggering settlement can receive a value that does not reflect the entity's true outstanding obligation and that changes the moment anyone calls `processRewards(...)`.

Recommendation(s): Consider revisiting the view so that the value it reports does not depend on whether the schedule has been settled recently, or documenting that it is exact only immediately after settlement.

Status: Fixed

Update from the client: Fixed (ed4f713). `_committedRewards` now measures the active entry's remainder at its settlement watermark (`scheduledValue - vested(lastUpdate)`) rather than the clock, so the view reports exactly the escrow still to leave `lockedRewardPool` regardless of when it is read; the queued entry continues to count in full until promoted. Immediately after settlement the value equals the previous one, so the internal escrow check is unchanged. Tests in `CommittedRewardsView.t.sol`, including a fuzz that `committedRewards ≤ lockedRewardPool` at every point, settled or not.

6.19 [Info] The configured minimum stake amount is never enforced

File(s): `contracts/vanaStaking/vanaPoolStaking/VanaPoolStakingImplementation.sol`

Description: The VanaPoolStaking contract stores a `minStakeAmount`, exposes it through the staking interface, and lets a maintainer change it through the `updateMinStakeAmount(...)` function, which emits an event announcing the new value. Every consumer of that interface would reasonably conclude that deposits below the configured amount are rejected.

No such check exists. The value is assigned during initialization and by the setter, and is never read anywhere in the staking logic. The only size constraint applied by the `stake(...)` function is that the deposit must mint at least one share.

```

1  uint256 vanaToShare = vanaPoolEntity.vanaToEntityShare(entityId);
2  uint256 sharesIssued = (vanaToShare * stakeAmount) / 1e18;
3
4  // @audit-issue This is the only size constraint; minStakeAmount is never read.
5  if (sharesIssued == 0) {
6      revert InsufficientStakeAmount();
7  }

```

At a share price near parity that threshold is satisfied by a deposit of one or two wei. On its own the consequence is limited to unbounded growth of the tracked staker set and of per-entity position storage, since the recipient of a deposit is an arbitrary address chosen by the caller. The more material consequence is that this dead parameter is the enabler for the reward stream capture and for the residual that triggers the unclamped decrement of distributed rewards, both reported separately.

Recommendation(s): Consider enforcing the configured minimum in the staking path, and reviewing whether the same floor should apply to the value moved by a redelegation. If the parameter is not intended to be enforced, consider removing it together with its setter and event, so that integrators are not misled into relying on a constraint the protocol does not apply.

Status: Fixed

Update from the client:

Update from Nethermind: fixed at [commit](#).

Update from Nethermind#2: the `minStakeAmount` is not enforced on the `unstake` && `redelegate` paths. A user can unstake or redelegate such that they will end up with a position that's below `minStakeAmount`.

Update from the client: Fixed (7752e34). `minStakeAmount` is now enforced on exit paths as well: a partial unstake/unstakeVana must leave nothing or a position worth at least the minimum, and redelegate must move at least the minimum and leave nothing or at least the minimum in the source. Full exits remain unrestricted. Registration floors exceed the minimum on both chains, so registrants are never trapped between the two rules. Tests in `MinStakeResidual.t.sol`.

Update from Nethermind#3 (Telegram): Now in legacy pools, if someone have a position which is worth less than `minShareAmount`, they won't be able to delegate their position and will need to unstake in order to stake in different pool. It can be intended design choice though.

Update from the client: Agreed and refined (7171d8b). A whole-position redelegation is now always allowed, so a legacy position below the minimum can still migrate (relocating it creates no dust); only splitting a position must respect `minStakeAmount` on both the moved and the residual side. Full exits remain unrestricted. Test `test_fullRedelegateOfASubMinimumLegacyPositionIsAllowed`.

6.20 [Info] The max amount quoted by getMaxUnstakeAmount(...) can be rejected by unstakeVana(...) after the minimum-stake-on-exit rule

File(s): `contracts/vanaStaking/vanaPoolStaking/VanaPoolStakingImplementation.sol`

Description: The `getMaxUnstakeAmount(...)` view is what an interface calls to populate a "max" button: it returns both `maxVana` (the most VANA a staker can take out) and `maxShares` (the same ceiling expressed in shares). For a reward eligible position it quotes the full value of the staker's shares.

```

1  uint256 userShareValue = (stakerEntity.shares * shareToVana) / 1e18;
2  // ...
3  } else {
4      // @audit Reward eligible: the quote is the full value of ALL the shares.
5      userWithdrawable = userShareValue;
6  }

```

A caller that then passes `maxVana` to `unstakeVana(...)` has the amount converted back to a share count with `vanaToShares(...)`, which floors. Because `maxVana` was itself produced by a flooring multiplication, the round trip lands a few share units below the staker's full holding, so the burn leaves a tiny remainder rather than clearing the position.

```

1  shareAmount = vanaPoolEntity.vanaToShares(entityId, vanaAmount);

```

The minimum-stake-on-exit rule added in the re-review then rejects that remainder, because it is worth far less than `minStakeAmount`.

```

1  uint256 remainingShares = stakerEntity.shares - shareAmount;
2  // @audit-issue The dust left by the maxVana round trip is below minStakeAmount,
3  // so the call quoted as the maximum reverts.
4  if (remainingShares > 0 && (remainingShares * shareToVana) / 1e18 < minStakeAmount) {
5      revert InsufficientStakeAmount();
6  }

```

The result is a mismatch between what the view promises and what the withdrawal accepts: for a matured position whose share price is not a whole number (which is any position that has accrued rewards), calling `unstakeVana(...)` with the quoted `maxVana` reverts with `InsufficientStakeAmount`. There is no impact on funds, and the staker can still exit in full by using the share denominated path with the quoted `maxShares`, or by lowering the requested amount so the remainder clears the minimum. The consequence is limited to the honesty of the `maxVana` figure on the VANA denominated withdrawal path: an integrator that wires the "max" button to `unstakeVana(maxVana)` will see it revert. This is the same class of view versus execution mismatch as the original maximum-unstake finding, reintroduced on the VANA path by the new exit rule.

Recommendation(s): Consider making the VANA denominated maximum consistent with what the withdrawal will accept, so that the quoted value either clears the position entirely or leaves at least the minimum, or documenting that the share denominated `maxShares` is the value to use for a full exit.

Status: Fixed

Update from the client: `unstakeVana` now serves a request that would strand a sub-minimum remainder as the full exit it amounts to, provided `shareAmountMax` admits the whole position (the `maxShares` returned alongside `maxVana`; 0 is not consent) and the registration floor allows it, so `unstakeVana(maxVana, maxShares, ...)` clears a matured position even against a slightly stale quote, while any other request keeps the existing exit rule and nothing is burned beyond the caller's bound.

6.21 [Info] Unstaking cost grows linearly with the number of entities ever created

File(s): /vanaPoolStaking/VanaPoolStakingImplementation.sol

Description: The VanaPoolStaking contract keeps an active and an inactive staker list. Adding a staker to the active list is a constant-time operation performed whenever a position is opened. Removing one, which happens on every single unstake, first determines whether the staker still holds a position anywhere, by scanning every entity identifier that has ever been issued.

```

1  function _removeStaker(address staker) internal {
2      uint256 entitiesCount = vanaPoolEntity.entitiesCount();
3      bool hasStake = false;
4      // @audit-issue This scans every entity ever created, on every unstake.
5      for (uint256 i = 1; i <= entitiesCount; i++) {
6          if (_stakers[staker].entities[i].shares > 0) {
7              hasStake = true;
8              break;
9          }
10     }
11     // ...
12 }
```

Each iteration reads a distinct cold storage slot, and the loop bound only ever increases, because the entity counter is never decremented and entity removal is commented out. The scan also runs on partial withdrawals where the position clearly survives, and the early exit only helps stakers whose lowest-numbered position is found quickly, so a staker whose only position is a recently created entity always pays the full scan.

The result is that the cost of leaving the protocol grows without bound as the protocol grows, and can never be reduced, while the cost of entering remains constant. Entity creation is restricted to a maintainer, so the counts at which withdrawal would become genuinely impossible are not attacker-reachable; the realistic impact is a permanent and steadily increasing burden on the one operation that must always remain available to users.

Recommendation(s): Consider tracking each staker's non-empty positions directly, for example with a set that is updated as positions are opened and closed, so that list membership can be determined without scanning the entire entity space. Consider also avoiding the check entirely on partial withdrawals where the position demonstrably survives.

Status: Fixed

Update from the client: Partially addressed (2b5849b). _removeStaker's scan now runs only on a full exit; a partial withdrawal leaves the position alive, so the staker is known to be active and the scan is skipped (behaviour unchanged, pinned by test_partialUnstake-KeepsStakerActive_fullExitDeactivates). Replacing the full-exit scan with a per-staker position count needs a storage slot and a migration for existing stakers; acknowledged and left as a follow-up.

6.22 [Best Practice] The receive() function doesn't check if msg.value = 0

File(s): ../RewardSplitterImplementation.sol

Description: The receive() function from the RewardSplitterImplementation contract is used to fund the splitter's distributable balance. However, it does not check if the sent msg.value is zero. This allows anyone to call the function without attaching any value and spam the contract to emit worthless Funded events.

```

1  /// @notice Fund the splitter's distributable balance.
2  receive() external payable {
3      // @audit no check for msg.value == 0
4      emit Funded(msg.sender, msg.value);
5  }
6
```

Recommendation(s): Consider emitting the event only if msg.value is > zero.

Status: Fixed

Update from the client: Fixed (2b5849b). receive() emits Funded only when msg.value > 0; a zero-value call is still accepted but emits nothing. Test: test_zeroValueFundingEmitsNoEvent (RewardSplitter.t.sol).

6.23 [Best Practice] The updateEntityPool(...) function contains a redundant role check

File(s): /vanaPoolEntity/VanaPoolEntityImplementation.sol

Description: The updateEntityPool(...) function from the VanaPoolEntityImplementation contract is used to update the entity stake information. The function uses the onlyRole(VANA_POOL_ROLE) modifier to restrict access, but the function's body contains the same check hasRole(VANA_POOL_ROLE, msg.sender) that reverts with NotAuthorized(). Because the modifier already enforces this requirement, the internal check is redundant.

```

1  function updateEntityPool(
2      uint256 entityId,
3      uint256 shares,
4      uint256 amount,
5      bool isStake
6  ) external override whenNotPaused onlyRole(VANA_POOL_ROLE) {
7      // @audit This check is redundant because of the onlyRole modifier.
8      if (!hasRole(VANA_POOL_ROLE, msg.sender)) {
9          revert NotAuthorized();
10     }
11     // ...
12 }
13

```

Recommendation(s): Consider removing the redundant role check from the function body.

Status: Fixed

Update from the client: Fixed (2b5849b). The in-body hasRole check was removed; onlyRole(VANA_POOL_ROLE) alone gates the function. Test: test_updateEntityPoolStillGatedWithoutTheRedundantCheck.

6.24 [Best Practice] The codebase contains unused error and event declarations

File(s): /vanaPoolEntity/VanaPoolEntityImplementation.sol, /vanaPoolStaking/VanaPoolStakingImplementation.sol

Description: The VanaPoolEntityImplementation and VanaPoolStakingImplementation contracts declare several errors and events that are never used.

```

1  // vanaPoolEntity/VanaPoolEntityImplementation.sol
2  error InvalidEntityId();
3  error EntityNameAlreadyExists();
4  error NameTooShort();
5  error StakersStillPresent();
6
7  // vanaPoolStaking/VanaPoolStakingImplementation.sol
8  event EntityStakeRegistered(uint256 indexed entityId, address indexed ownerAddress);
9  error InsufficientShares();
10 error EntityNotFound();
11 error NotEntityOwner();
12 error CannotRemoveRegistrationStake();
13 error NotAuthorized();
14

```

Recommendation(s): Consider removing the unused error and event declarations from the contracts.

Status: Fixed

Update from the client: Fixed (2b5849b, c2b250c). Removed: VanaPoolEntity InvalidEntityId, EntityNameAlreadyExists, NameTooShort, StakersStillPresent, NotAuthorized (its only use was the redundant check below); VanaPoolStaking event EntityStakeRegistered, InsufficientShares, EntityNotFound, NotEntityOwner, NotAuthorized. Two names on your list are now in use and were kept: CannotRemoveRegistrationStake (raised by the registration floor) and VanaPoolEntity's NotEntityOwner.

6.25 [Best Practices] The calculateYield(...) interface declares its first two parameters in the wrong order

File(s): /vanaPoolEntity/interfaces/IVanaPoolEntity.sol

Description: The VanaPoolEntity contract exposes a pure helper that integrators can call to reproduce the protocol's yield calculation. The interface and the implementation disagree about what the first two arguments mean.

```

1 // @audit-issue The interface declares apy first and principal second.
2 function calculateYield(
3     uint256 apy,
4     uint256 principal,
5     uint256 time
6 ) external pure returns (uint256);

```

The implementation takes them in the opposite order, as principal then apy. All three parameters are uint256, so the selector is identical, the override compiles silently, and the single in-repository caller passes them positionally in the implementation's order and is therefore correct. Any integrator coding against the published interface will transpose the two values and receive a meaningless result, with no revert to signal it.

Recommendation(s): Consider renaming the interface parameters so that they match the implementation, and reviewing the other interface declarations for the same kind of drift.

Status: Fixed

Update from the client: Fixed (2b5849b). Interface renamed to (principal, apy, time) to match the implementation. Per your recommendation we scanned all three interfaces for the same drift and aligned the two other mismatches (IVanaPoolStaking.unstake's share parameter was named amount; IVanaPoolEntity.updateVanaPool's parameter name).

6.26 [Best Practices] The buy and burn sink can be cleared while a reserve is still outstanding

File(s): /rewardSplitter/RewardSplitterImplementation.sol

Description: The RewardSplitter contract sets aside a configurable fraction of each distributed budget in pendingBurn, which is later flushed to the configured sink. The updateBuyAndBurn(...) function requires a non-zero sink only when the rate itself is non-zero.

```

1 // @audit-issue The sink may be set to the zero address whenever the rate is
2 // @audit-issue zero, including while pendingBurn is still non-zero.
3 if (newBurnRate > 0 && newBuyAndBurnAddress == address(0)) {
4     revert InvalidAddress();
5 }

```

Disabling the feature by passing a zero rate and a zero sink is therefore accepted even when a reserve has already accrued. In that state the flush reverts on its own zero address check, while both the distribution and the withdrawal paths subtract pendingBurn from the balance they are allowed to touch, so the reserve is simultaneously unspendable and unwithdrawable. It is recoverable by re-configuring a sink, so this is a configuration trap rather than a loss.

Recommendation(s): Consider requiring a non-zero sink whenever a reserve is outstanding, regardless of the rate being set, so that the feature cannot be switched off in a way that strands funds already set aside.

Status: Fixed

Update from the client:

Removed the configurable sink entirely instead of guarding it. The cut now burns directly to address(0):

buyAndBurnAddress and updateBuyAndBurn(rate, addr) (with the flagged zero-address check) removed. Added BURN_ADDRESS = address(0); updateBurnRate(rate) sets only the rate; executeBurn() sends pendingBurn to BURN_ADDRESS unconditionally (permissionless). The trap is gone: no sink to misconfigure, and pendingBurn is always flushable via executeBurn() regardless of rate — so it can't be stranded.

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Vana Protocol's documentation

Throughout the engagement, Vana's team responded to Nethermind's security team's questions during sync calls and async communication channels.

The repository also included a README file which contained details about the architecture, components, and functionality of Vana's staking mechanism.

8 Test Suite Evaluation

8.1 Tests Output

```
forge test
Ran 5 tests for test/foundry/DepositContractSeeded.t.sol:OwnershipTest
[PASS] test_depositsUnaffectedByRenounce() (gas: 137017)
[PASS] test_onlyPendingOwnerCanAccept() (gas: 43697)
[PASS] test_renounceClearsPendingTransfer() (gas: 35174)
[PASS] test_renounceLeavesNoOwnerAndFreezesAdmin() (gas: 18717)
[PASS] test_twoStepTransfer() (gas: 35849)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 3.70ms (1.27ms CPU time)

Ran 8 tests for test/foundry/EntityCommission.t.sol:EntityCommissionTest
[PASS] test_apySkinSplitsAndConserves() (gas: 290864)
[PASS] test_hundredPercentCommission_delegatorsGetNothing() (gas: 270198)
[PASS] test_maintainerCanSetCommission() (gas: 243746)
[PASS] test_onlyOwnerOrMaintainerSetsCommission() (gas: 198727)
[PASS] test_rejectsAboveMaxCommission() (gas: 194282)
[PASS] test_streamSkinSplits() (gas: 329039)
[PASS] test_updateCommission_settlesAtOldRateFirst() (gas: 292799)
[PASS] test_zeroCommission_allToDelegators() (gas: 250602)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 3.28ms (1.64ms CPU time)

Ran 10 tests for test/foundry/EntityRewardModelSwitch.t.sol:EntityRewardModelSwitchTest
[PASS] test_nonActiveEntityReverts() (gas: 212248)
[PASS] test_onlyMaintainerCanSwitch() (gas: 197881)
[PASS] test_switchApyToStream_settlesAndConserves() (gas: 274900)
[PASS] test_switchStreamToApy_vestsThenFlips() (gas: 273482)
[PASS] test_switchToSameModel_isNoOpBeyondSettle() (gas: 254144)
[PASS] test_switchToStreamModel_clearsStaleSchedule() (gas: 273364)
[PASS] test_switchToStreamModel_rejectsNonApy() (gas: 235792)
[PASS] test_switchToStreamModel_rollsResidueIntoStream() (gas: 301942)
[PASS] test_switchToStreamModel_thenVestsLinearly() (gas: 321406)
[PASS] test_switchToStreamModel_zeroResidue_parksInStreamNoSchedule() (gas: 231007)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 4.81ms (2.79ms CPU time)

Ran 9 tests for test/foundry/DistributeRewards.t.sol:DistributeRewardsTest
[PASS] test_allowsSchedulingFromResidue_msgValueZero() (gas: 243898)
[PASS] test_installsActiveWhenNoneExists() (gas: 252979)
[PASS] test_onlyOwnerOrMaintainer() (gas: 200665)
[PASS] test_queuesWhenStartsAtOrAfterActiveEnd() (gas: 271928)
[PASS] test_rejectsBackdatedStartAndZeroAmount() (gas: 202063)
[PASS] test_replaceOnOverlap_cancelsRemainderToResidue() (gas: 301799)
[PASS] test_requiresStreamModel() (gas: 193636)
[PASS] test_revertsWhenUnderfunded() (gas: 239891)
[PASS] test_scheduledStreamVestsThroughProcessRewards() (gas: 298253)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 4.90ms (2.77ms CPU time)

Ran 9 tests for test/foundry/EntityRewardViews.t.sol:EntityRewardViewsTest
[PASS] test_committedRewards_activeUnvestedPlusQueued() (gas: 234536)
[PASS] test_committedRewards_zeroForApyEntity() (gas: 167674)
[PASS] test_currentAPY_apyMode_matchesCapWhenFunded() (gas: 193282)
[PASS] test_currentAPY_apyMode_zeroWhenNoLocked() (gas: 168432)
[PASS] test_currentAPY_streamMode_annualizedRate() (gas: 231990)
[PASS] test_currentAPY_streamMode_zeroOutsideWindow() (gas: 234101)
[PASS] test_currentAPY_streamMode_zeroWhenNoActivePool() (gas: 211605)
[PASS] test_rewardModelView() (gas: 168479)
[PASS] test_rewardScheduleView() (gas: 229973)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 4.12ms (1.93ms CPU time)

Ran 4 tests for test/foundry/DepositContractSeeded.t.sol:MinDepositAmountTest
[PASS] test_boundaryIsInclusive() (gas: 179101)
[PASS] test_nonOwnerCannotUpdate() (gas: 13861)
[PASS] test_ownerCanUpdate_withEvent() (gas: 21515)
[PASS] test_updateAppliesToSubsequentDepositsOnly() (gas: 183072)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 5.84ms (2.96ms CPU time)

Ran 2 tests for test/foundry/DepositContractSeeded.t.sol:MainnetSeedTest
[PASS] test_depositsContinueFromMainnetState() (gas: 2053526)
[PASS] test_reproducesMainnetRootAndCount() (gas: 2016692)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 5.93ms (2.35ms CPU time)

Ran 5 tests for test/foundry/DepositContractSeeded.t.sol:ConstructorTest
[PASS] test_emptyDeploymentMatchesCanonicalEmptyRoot() (gas: 1828424)
[PASS] test_storageLayoutMatchesCanonicalContract() (gas: 1880326)
[PASS] test_storesConstructorArguments() (gas: 1935035)
[PASS] test_supportsInterface() (gas: 1797259)
[PASS] test_zeroOwnerReverts() (gas: 44589)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 6.61ms (4.51ms CPU time)
```

```
Ran 11 tests for test/foundry/DepositContractSeeded.t.sol:RestrictedModeTest
[PASS] test_allowedKeyMayDepositRepeatedly() (gas: 277329)
[PASS] test_allowlistIgnoredWhileOpen() (gas: 173946)
[PASS] test_batchAddRemove_withEvents() (gas: 84796)
[PASS] test_controlsAreOwnerOnly() (gas: 34093)
[PASS] test_deployAlreadyRestrictedWithSeedList() (gas: 1859938)
[PASS] test_disablingRestrictionReopensDeposits() (gas: 144247)
[PASS] test_openByDefault() (gas: 135806)
[PASS] test_removeRevokesAccess() (gas: 122137)
[PASS] test_restrictedBlocksUnlistedKey() (gas: 105419)
[PASS] test_restrictedPermitsAllowedKeyOnly() (gas: 246155)
[PASS] test_updateRestrictedEmitsEvent() (gas: 37694)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 7.49ms (6.50ms CPU time)

Ran 7 tests for test/foundry/RewardModelsE2E.t.sol:RewardModelsE2ETest
[PASS] test_apyModel_stakerEarnsAndWithdraws() (gas: 792007)
[PASS] test_commission_delegatorEarnsPostCut_ownerClaims() (gas: 868246)
[PASS] test_erc20RewardFlow_convertAndDistribute() (gas: 1442390)
[PASS] test_launchHandoff_cappedThenRollToStream() (gas: 843578)
[PASS] test_streamBonding_earlyExitForfeitsRewards() (gas: 861551)
[PASS] test_streamBonding_lateExitCollectsRewards() (gas: 846225)
[PASS] test_streamModel_stakerEarnsAndWithdraws() (gas: 828827)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 8.50ms (8.28ms CPU time)

Ran 7 tests for test/foundry/Redelegation.t.sol:RedelegationTest
[PASS] test_redelegate_completeBondInB_keepsRewards() (gas: 1251210)
[PASS] test_redelegate_earlyExitFromB_forfeitsRewardPortion() (gas: 1230391)
[PASS] test_redelegate_mergesIntoExistingPosition_weightsBond() (gas: 1262956)
[PASS] test_redelegate_midBond_carriesBondAndRewards() (gas: 1225402)
[PASS] test_redelegate_movesValueNoVanaLeavesTreasury() (gas: 1196370)
[PASS] test_redelegate_rejectsMoreThanOwned() (gas: 966374)
[PASS] test_redelegate_rejectsSameEntity() (gas: 648905)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 8.09ms (10.91ms CPU time)

Ran 8 tests for test/foundry/EntityStakeBlock.t.sol:EntityStakeBlockTest
[PASS] test_blockRejectsExistingStakerTopUp() (gas: 680672)
[PASS] test_blockRejectsNewStaker() (gas: 522707)
[PASS] test_maintainerCanBlock() (gas: 492834)
[PASS] test_redelegateIntoBlockedEntityRejected() (gas: 989518)
[PASS] test_redelegateOutOfBlockedEntityWorks() (gas: 1060541)
[PASS] test_strangerCannotBlock() (gas: 471177)
[PASS] test_unblockRestoresStaking() (gas: 654682)
[PASS] test_unstakeStillWorksWhileBlocked() (gas: 693361)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 9.77ms (10.03ms CPU time)

Ran 5 tests for test/foundry/TopUpQueuedRewards.t.sol:TopUpQueuedRewardsTest
[PASS] test_revertsForApyModel() (gas: 222276)
[PASS] test_revertsForNonOwner() (gas: 266642)
[PASS] test_revertsOnZeroValue() (gas: 255453)
[PASS] test_revertsWhenNothingQueued() (gas: 242403)
[PASS] test_topUpGrowsQueuedEntryAndFundsTreasury() (gas: 294163)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 6.32ms (1.32ms CPU time)

Ran 17 tests for test/foundry/RewardSplitter.t.sol:RewardSplitterTest
[PASS] test_burnAccruesAndEntitiesGetRemainder() (gas: 1372647)
[PASS] test_burnRateCapAndSinkRequired() (gas: 30459)
[PASS] test_executeBuyAndBurnFlushesToSink() (gas: 1388597)
[PASS] test_firstRoundSetsBaselineNoPayout() (gas: 1198648)
[PASS] test_flashStakeGetsNegligibleShare() (gas: 1368366)
[PASS] test_fundTokenRewardPullsAndRecords() (gas: 560834)
[PASS] test_fundTokenRewardRejectsZeroTokenAndAmount() (gas: 518033)
[PASS] test_noBurnWhenUnconfigured() (gas: 1328459)
[PASS] test_onlyDistributorCanDistribute() (gas: 782375)
[PASS] test_recoverTokenOnlyExcessNotRecorded() (gas: 609107)
[PASS] test_rejectsBudgetOverBalance() (gas: 789774)
[PASS] test_returnedVanaIsDistributable() (gas: 591157)
[PASS] test_sendToConverterForwardsAndDecrements() (gas: 623400)
[PASS] test_sendToConverterGuards() (gas: 612092)
[PASS] test_splitsProportionalToStakeSecondsDelta() (gas: 1379489)
[PASS] test_updateConverterAccessAndZeroAddress() (gas: 32085)
[PASS] test_withdrawCannotTouchPendingBurn() (gas: 1377431)
Suite result: ok. 17 passed; 0 failed; 0 skipped; finished in 11.85ms (20.21ms CPU time)

Ran 5 tests for test/foundry/StakeSeconds.t.sol:StakeSecondsTest
[PASS] test_firstTouchStartsClockNoPhantom() (gas: 178102)
[PASS] test_linearAccrual() (gas: 173956)
[PASS] test_nonActiveFrozen() (gas: 175278)
[PASS] test_staircaseMatchesHandComputedIntegral() (gas: 200841)
[PASS] test_viewEqualsStoredAfterCheckpoint() (gas: 196431)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 15.29ms (1.62ms CPU time)
```

```
Ran 8 tests for test/foundry/VestStream.t.sol:VestStreamTest
[PASS] testFuzz_totalVestedNeverExceedsValue(uint32,uint8) (runs: 256, : 61516, ~: 58732)
[PASS] test_clampsAtEnd_totalEqualsValueExactly() (gas: 55056)
[PASS] test_instantEntry_vestsFullValueOnce() (gas: 37950)
[PASS] test_nothingBeforeStart() (gas: 40164)
[PASS] test_queuedEntryPromotesAndVests() (gas: 50676)
[PASS] test_timingInvariance_oneStepEqualsManySteps() (gas: 77643)
[PASS] test_vestsLinearlyOverDuration() (gas: 45642)
[PASS] test_zeroShares_returnsZeroAndPreservesInterval() (gas: 42951)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 68.29ms (67.96ms CPU time)

Ran 10 tests for test/foundry/DepositContractSeeded.t.sol:DepositValidationTest
[PASS] testFuzz_acceptsAnyValidAmount(uint256) (runs: 256, : 130082, ~: 130105)
[PASS] test_emitsDepositEvent() (gas: 140092)
[PASS] test_rejectsBadPubkeyLength() (gas: 40713)
[PASS] test_rejectsBadSignatureLength() (gas: 38717)
[PASS] test_rejectsBadWithdrawalCredentialsLength() (gas: 34975)
[PASS] test_rejectsNonGweiMultiple() (gas: 37707)
[PASS] test_rejectsValueAboveUint64Gwei() (gas: 38202)
[PASS] test_rejectsValueBelowMinimum() (gas: 37479)
[PASS] test_rejectsWrongDepositDataRoot() (gas: 83907)
[PASS] test_validDepositUpdatesState() (gas: 252473)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 103.35ms (105.43ms CPU time)

Ran 4 tests for test/foundry/DepositContractSeeded.t.sol:SeedingTest
[PASS] testFuzz_seedReproducesRoot(uint8,uint64) (runs: 256, : 4262481, ~: 4272461)
[PASS] test_countAloneDoesNotReproduceRoot() (gas: 3929235)
[PASS] test_firstEventIndexContinuesFromSeed() (gas: 4104385)
[PASS] test_seedReproducesRootAndStaysInLockstep() (gas: 4735355)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 569.16ms (585.90ms CPU time)

Ran 18 test suites in 582.54ms (847.31ms CPU time): 134 tests passed, 0 failed, 0 skipped (134 total tests)
```

8.2 Automated Tools

8.2.1 AuditAgent

The AuditAgent is an AI-powered smart contract auditing tool that analyses code, detects vulnerabilities, and provides actionable fixes. It accelerates the security analysis process, complementing human expertise with advanced AI models to deliver efficient and comprehensive smart contract audits. Available at <https://app.auditagent.nethermind.io>.

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.